

Backstab 개발 보고서

Claude Code와의 반복 협업으로 만든 소셜 디덕션 파티게임

프로젝트 : Backstab (백스탭) — 느와르 조직 테마 소셜 디덕션 파작성일 : 2026-08-16

티게임

기술 스택 : Node.js · TypeScript · Express · Socket.IO / 순수

배포 : <https://backstab-tu0e.onrender.com> (Render)

HTML·CSS·JS 프론트엔드

개발 기간 : 2026-08-05 ~ 2026-08-16(진행 중), 커밋 80건 이상

개발 방식 : Claude Code(CLI 에이전트)와의 한국어 대화형 반복 협업

1. 개요

Backstab은 보스·조직원·스파이·배신자 네 진영이 낮의 투표와 밤의 은밀한 행동을 주고받으며 서로의 정체를 추리하는 소셜 디덕션 파티게임이다. 이 보고서의 초점은 게임 자체의 규칙이 아니라 **만들어진 과정**에 있다. 기획 문서 작성부터 서버 로직, UI 구현, Render 배포, 실사용 중 발견된 버그 수정까지 전 과정이 별도의 이슈 트래커나 스펙 문서 없이 사용자와 Claude Code의 한국어 대화형 협업만으로 진행됐다. "지목했는데 두 명이 선택된다"처럼 형식을 갖추지 않은 구두 제보가 코드 조사와 수정으로 이어지는 사이클이 열흘 남짓 반복되며 게임이 완성됐다.

2. 협업 워크플로우 — 지시가 아니라 위임

사용자는 대부분 결과 화면이나 증상만 전달했고, 원인을 지목해준 경우는 거의 없었다. Claude Code는 매번 (1) 관련 코드 경로 탐색 → (2) 재현 조건 특정 → (3) 가설 검증 → (4) 수정 → (5) 유닛 테스트·실제 소켓 통신·스크린샷으로 검증 → (6) 개발로그 기록의 순서로 스스로 대응했다. 예컨대 "클릭하면 카드가 두 명씩 선택된다"는 제보는 표면적으로는 CSS 버그처럼 보였지만, 실제로는 서버가 같은 소켓의 재입장을 막지 않아 플레이어 레코드가 중복 생성되는 문제였다(3장 (a) 참고) — 사용자의 원인 진단이 아니라 Claude Code가 코드를 끝까지 추적해 도달한 결론이었다.

3. 근본 원인을 파고든 디버깅 사례

초기 가설이 틀렸는데도 거기서 멈추지 않고 실제 원인까지 추적한 대표 사례 4건이다.

사례 (a) 중복 소켓 입장 — 플레이어 이중 등록

증상: 한 명을 지목했는데 카드 두 장이 동시에 선택된 것처럼 보이고, 진행자 화면 투표 집계도 어긋남.

원인: `player:join_room` 핸들러가 닉네임 중복만 검사하고 같은 소켓의 재입장은 막지 않았다. 같은 `socket.id`를 가진 Player 레코드가 두 개 생성되면서, 이 값이 게임 전체의 식별자로 쓰이는 탓에 지목·투표·집계가 매번 두 카드에 겹쳐 반영됐다.

해결: `addPlayer()`에 "한 소켓 = 한 자리" 불변식을 함수 내부에 못박아 모든 호출 지점이 함께 보호되도록 했다. 회귀 테스트 2건을 추가하고 실제 소켓으로 3회 연속 재입장을 시도해 1명만 등록되는 것까지 확인했다.

사례 (b) JS 임시 사각지대(TDZ) 크래시 — 결과 화면이 조용히 무너짐

증상: 게임이 끝났는데 승리 슬라이드가 사라진 뒤 최종 결과 화면이 뜨지 않음.

원인: `applyPhase()`의 `game_over` 분기가 참조하는 `const` 선언 3개가 함수 뒤쪽에 있어 "Cannot access before initialization" 예외가 발생, 함수 실행이 그 자리에서 끊겼다. 마침 그 위를 승리 슬라이드가 가리고 있어 화면상으로는 크래시가 드러나지 않는 **조용한 실패**였다 — 콘솔을 열어보지 않으면 발견할 수 없는 유형.

해결: 세 선언을 함수 앞쪽으로 옮겨 `game_over` 분기보다 먼저 초기화되게 했다.

사례 (c) 방장 수동 진행이 남긴 서버 타이머 누수

증상: 알 수 없는 시점에 투표·밤 처리가 한 번 더 실행되며 몇 라운드 지난 게임 상태가 갑자기 어긋남.

원인: 방장이 `host:advance_phase` 로 다음 단계를 수동으로 넘겨도, 자동 진행을 위해 심어둔 `phaseTimer` 가 지워지지 않고 백그라운드에 그대로 남아 있었다. 이 타이머는 원래 예정된 시각에 몇 라운드나 지난 방 상태에 대해 `resolveVote` 등을 다시 호출해, 방장이 "다음 단계로"를 누를 때마다 사실상 매번 새로고쳐 있었다.

해결: 수동 진행 처리 시 예약된 타이머를 함께 제거하도록 수정.

사례 (d) 자동 재접속과 새 방 생성 사이의 경쟁 상태

증상: 새 방을 만들었는데 방 코드가 화면에 안 뜨거나, 방장 전용 버튼이 사라진 채로 남음.

원인: 저장된 세션으로 자동 `player:reconnect` 를 시도하는 흐름과, 그 응답이 오기 전에 사용자가 누른 "새 방 만들기" 흐름이 동시에 진행됐다. 세션을 지우는 처리는 이후 저장될 세션만 막을 뿐 이미 날아간 재접속 요청은 취소하지 못해, 나중에 도착한 응답이 방금 만든 새 방 상태를 조용히 덮어썼다.

해결: "새 방 만들기" 클릭 시점에 플래그를 세워 이후 도착하는 모든 재접속 응답(대기 중이던 것 포함)을 무시하도록 수정.

4. 프로젝트 전용 Claude Code 스킬 라이브러리

반복되는 작업을 `.claude/skills/` 아래 재사용 가능한 스킬로 정형화해 축적했다.

스킬	역할
게임 QA	사용자가 실제 플레이어(또는 호스트)로 참여하면 나머지 자리를 소켓 봇으로 채우고, 게임이 시작된 뒤로는 다시 묻지 않고 종료까지 밤 행동·투표를 자동으로 채워 진행한다.
자동 QA	사용자 없이 배포판(Render)에서 혼자 여러 판을 완주한다. 승리조건 3종(스파이·보스측·배신자)을 각각 강제로 만들고, 인원 6~10명과 화면 3종 크기(데스크톱·아이패드·폰)를 점검한 뒤 문제를 고치고 로그·커밋까지 마무리한다.
화면 확인	UI에 영향을 주는 파일을 수정한 직후 요청 없이도 브라우저를 띄워 스크린샷을 찍어 검증한다.
개발 로그	세션마다 실제로 완료한 작업을 <code>docs/10개발로그.md</code> 에 날짜별·시간순으로 원인/해결 형태로 누적 기록한다.
커밋 푸시	변경사항을 확인하고 <code>git add-commit-push</code> 를 자동으로 진행한다.

특히 **자동QA**는 Claude Code가 스스로의 QA 프로세스를 확장하기 위해 사용자 요청으로 직접 설계·작성한 스킬이라는 점이 특징적이다 — 에이전트가 자신의 다음 작업을 위한 도구를 스스로 만든 사례다.

5. 새로운 테스트 방법론 — 순수 유닛 테스트를 넘어서

자동QA 스킬은 브라우저 탭 하나에서 `socket.io` 클라이언트를 여러 개 열어 봇 여러 명 분의 소켓을 동시에 굴리는 하네스를 쓴다. 8개 탭을 여는 방식보다 훨씬 안정적이고 빠르다는 것이 실전에서 확인됐다. 이 하네스로 다음을 실사용 환경에서 검증한다.

- 역할을 실시간으로 읽어(`__roles()`) 원하는 승리 조건(스파이·보스측·배신자 승리)을 의도적으로 강제하며 **실제 배포판**에서 한 판을 통째로 완주시킨다.
- Playwright로 실제 브라우저 UI를 조작해 데스크톱(1920×1080)·아이패드(1180×820 / 820×1180 세로)·폰(390×844) 세 화면 크기에서 카드 잘림·가로 스크롤·타이머 넘침 여부를 **수치로** 측정한다(눈으로만 보지 않는다).
- 재접속, 세션 경쟁 조건처럼 타이밍에 의존하는 버그는 순수 유닛 테스트로는 구조적으로 재현되지 않고, 실제 소켓 이벤트를 순서대로 흘려보내야만 드러난다는 것을 이 방식으로 실전 확인했다. 실제로 배신자 승리 분기는 판정 순서 문제로 **어떤 입력으로도 도달할 수 없는 죽은 코드**였는데, 이를 소켓 하네스로 실제 배포판에서 재현·검증한 뒤에야 고칠 수 있었다.

6. 테스트 하네스 버그와 실제 제품 버그의 구분

자동QA 스킴 문서에는 "하네스 문제와 제품 버그를 반드시 구분해서 보고한다"는 원칙이 명시돼 있다. 실제로 봇 3명이 밤에 동시에 공격을 제출했는데 데미지가 1로만 집계된 사례가 있었다. 원인은 게임 로직이 아니라 **테스트 하네스**였다 — 제출 직후 곧바로 다음 단계 진행 명령을 보내는 바람에 서버에 진행 신호가 제출보다 먼저 도착해 나머지 제출이 유실된 것이었다. 제출과 진행 사이에 텀을 넣어 해결했고, 이 교훈은 스킴 문서에 "반드시 지킬 것"으로 못박혀 이후 QA에서 같은 오진(없는 버그를 찾는 일)을 반복하지 않도록 했다.

이 구분 원칙이 없었다면 위 사례는 "밤 공격 합산 로직 버그"로 잘못 보고되어 존재하지 않는 버그를 찾는 데 시간이 쓰였을 것이다.

7. 재현 안 되는 버그를 재현됐다고 하지 않기, 애매한 결정은 질문하기

이후 세션에서 사용자는 실 플레이 중 겪은 문제 4건을 한 번에 보고했다 — 호스트 화면의 투표완료 표시 누락, 보스 "긴급 처형"의 중복 사용, 스파이끼리의 실시간 선택 확인 불가, 그리고 긴급 처형의 효과 자체를 "HP와 무관하게 즉시 처형"으로 바꿔달라는 요청. Claude Code는 넷 다 곧바로 "고쳤습니다"라고 답하는 대신 실제 소켓 통신과 실 UI로 하나씩 재현을 먼저 시도했다.

앞의 세 가지는 코드를 정적으로 검토해도, 6인 게임을 실제로 완주시켜도 재현되지 않았다 — 6명 전원이 투표하자 호스트 화면에 제출 배지가 모두 정상적으로 떴고, 긴급 처형은 실 UI로 한 번 쓴 뒤 다음 라운드 선택지에서 정확히 사라졌으며, 스파이의 대상 선택도 제출 전부터 다른 스파이에게 "(고르는 중)" 상태로 실시간 전달됐다. Claude Code는 원인을 지어내는 대신, 세션 초반에 이미 고친 서버 타이머 누수(사례 c) — 방장이 페이지를 수동으로 넘기면 남아있던 타이머가 나중에 엉뚱한 시점에 재발화해 투표·능력 상태를 조용히 흐트러뜨리던 버그 — 가 세 증상의 공통 원인이었을 가능성이 높다고, 재현 결과와 함께 있는 그대로 보고했다.

네 번째 요청은 실제 기능 변경이었다. "육탄방어로 막을 수 있어야 하는지"는 밸런스에 직결되는 설계 판단이라, Claude Code는 스스로 정하는 대신 AskUserQuestion으로 사용자에게 직접 물어 답을 받은 뒤에야 구현에 들어갔다. 구현은 데미지 수치를 올리는 방식 대신 "대상의 현재 HP를 그대로 raw damage로 먹인다"는 방식을 택해 기존 방어(전량 흡수/절반 경감) 파이프라인과 자연스럽게 맞물리게 했고, 유닛 테스트 2건과 실제 소켓 흐름(HP 4 대상이 한 방에 사망)으로 검증한 뒤 배포했다.

8. 문서화 — 세션을 넘어서는 기억

`docs/10개발로그.md`는 날짜별·시간순으로 **실제로 완료된 작업만** 원인과 해결 방식을 담아 누적 기록하며, 진행 중이거나 계획 단계인 항목은 별도 메모 문서로 분리해 섞이지 않게 한다. 기존 항목은 수정·삭제하지 않고 덧붙이기만 한다는 규칙 덕분에, 다음 세션의 Claude Code는 이전 대화를 기억하지 못해도 이 문서 하나로 프로젝트 이력 전체를 파악하고 곧바로 이어서 작업할 수 있다. 단발성 대화가 아니라 누적되는 개발 조직처럼 프로젝트가 굴러가게 하는 장치다.

결론

Backstab은 열흘 남짓한 기간 동안 80건 이상의 커밋으로 기능 구현과 버그 수정을 반복하며 완성됐다. 이 속도를 지탱한 것은 Claude Code가 지시를 문자 그대로 수행하는 데 그치지 않고, 증상 이전의 근본 원인을 스스로 추적하고(3장), 반복 작업을 스킴로 재사용 가능하게 만들고(4장), 실제 배포 환경을 대상으로 한 새로운 테스트 방식을 직접 고안해 적용했으며(5·6장), 재현되지 않는 제보를 재현됐다고 꾸미지 않고 애매한 설계 판단은 추측 대신 되물었다는 점이다(7장). 사용자는 무엇이 이상한지를 알려주는 역할에 가까웠고, 원인 규명부터 구현·검증·기록까지 이어지는 나머지 루프는 에이전트가 스스로 완결했다.